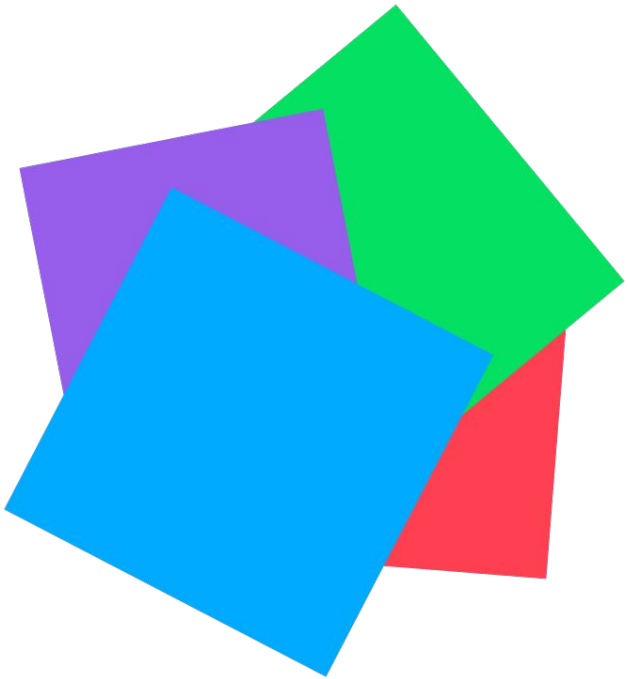
ООП и принципы программирования



Avito Start

Рассказывает Яковлев Сергей

Engineer / Monetization

- Выпускник лучшего вуза страны,
  на это можно выделить две строки
- Опыт в Авито: большие данные и огромные данные
- Носит черную кепку и зеленую кофту тоже можно в две строки
- Супер интересный факт под номером четыре, без него никак!



Avito Start

О чем поговорим

1. Базовые принципы ООП: инкапсуляция, наследование (композиция), полиморфизм

2. SOLID и его применение в GO

3. GRASP шаблоны

4. Другие принципы программирования



Avito Start

Базовые принципы ООП



Популярные подходы

1. Процедурный

Делим задачу на процедуры, работающие с данными

2. Функциональный

Вычисления из функций и преобразований данных

3. Объектно-ориентированный

Разделить состояние и поведение между объектами

Принципы ООП



Инкапсуляция

Объект управляет своим состоянием



Наследование

Новый тип строится на основе существующего

В Go классического наследования типов **нет**



Полиморфизм

Разные объекты выполняют одну операцию по-разному



Абстракция

Выделяем важные свойства объекта

Абстракция

Абстракция выделяет свойства и действия, важные для задачи, оставляя детали реализации за её границами.

Интерфейс - один из способов выразить абстракцию в Go

```
type Car interface {
	Drive(distanceKm int)
}

// Реализация с бензиновым двигателем.
type PetrolCar struct{}

func (PetrolCar) Drive(distanceKm int) {
	fmt.Printf("Бензиновый двигатель: едем %d км\n", distanceKm)
}

// Реализация с электромотором.
type ElectricCar struct{}

func (ElectricCar) Drive(distanceKm int) {
	fmt.Printf("Электромотор: едем %d км\n", distanceKm)
}
```

Полиморфизм

Travel принимает Car и вызывает Drive().

Один и тот же код работает с PetrolCar и ElectricCar. Конкретная реализация определяет поведение.

```
type Car interface { Drive(distanceKm int) }
type PetrolCar struct{}
func (PetrolCar) Drive(km int) {
  fmt.Printf("Бензин: едем %d км\n", km)
}
type ElectricCar struct{}
func (ElectricCar) Drive(km int) {
  fmt.Printf("Электромотор: едем %d км\n", km)
}

func Travel(car Car) {
  car.Drive(10)
}
func main() {
  Travel(PetrolCar{})
  Travel(ElectricCar{})
}
```

Наследование и композиция

Наследование позволяет создать новый тип на основе существующего, используя и расширяя его свойства и поведение.

Композиция позволяет построить объект из других объектов и использовать их возможности.

```
type Engine struct{}

func (engine Engine) Start() {
  fmt.Println("Двигатель запущен")
}

type PetrolCar struct {
  engine Engine
}

func (car PetrolCar) Start() {
  car.engine.Start()
}
```

Инкапсуляция

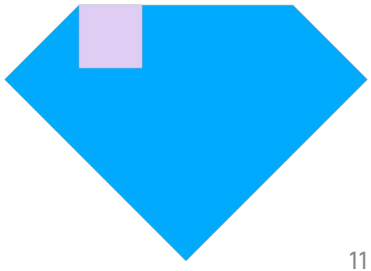
Инкапсуляция объединяет состояние и поведение объекта и контролирует доступ к внутренним данным.

```
type PetrolCar struct {
  speed int
}

func (car *PetrolCar) SetSpeed(speed int) error {
  if speed < 0 || speed > 180 {
    return fmt.Errorf("недопустимая скорость: %d", speed)
  }

  car.speed = speed
  return nil
}
```

Можно ли назвать Go ООП языком?



Разберите на принципы ООП*

```
type Pizza interface {
    Price() int
}

type Margherita struct{}
func (Margherita) Price() int { return 600 }

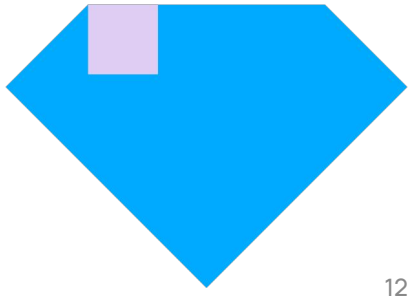
type Pepperoni struct{}
func (Pepperoni) Price() int { return 750 }

type orderItem struct {
    pizza Pizza
    count int
}

type Order struct {
    items []orderItem
}
```

```
func (o *Order) Add(pizza Pizza, count int) error {
    if count <= 0 {
        return errors.New("количество должно быть
положительным")
    }
    o.items = append(o.items, orderItem{pizza, count})
    return nil
}

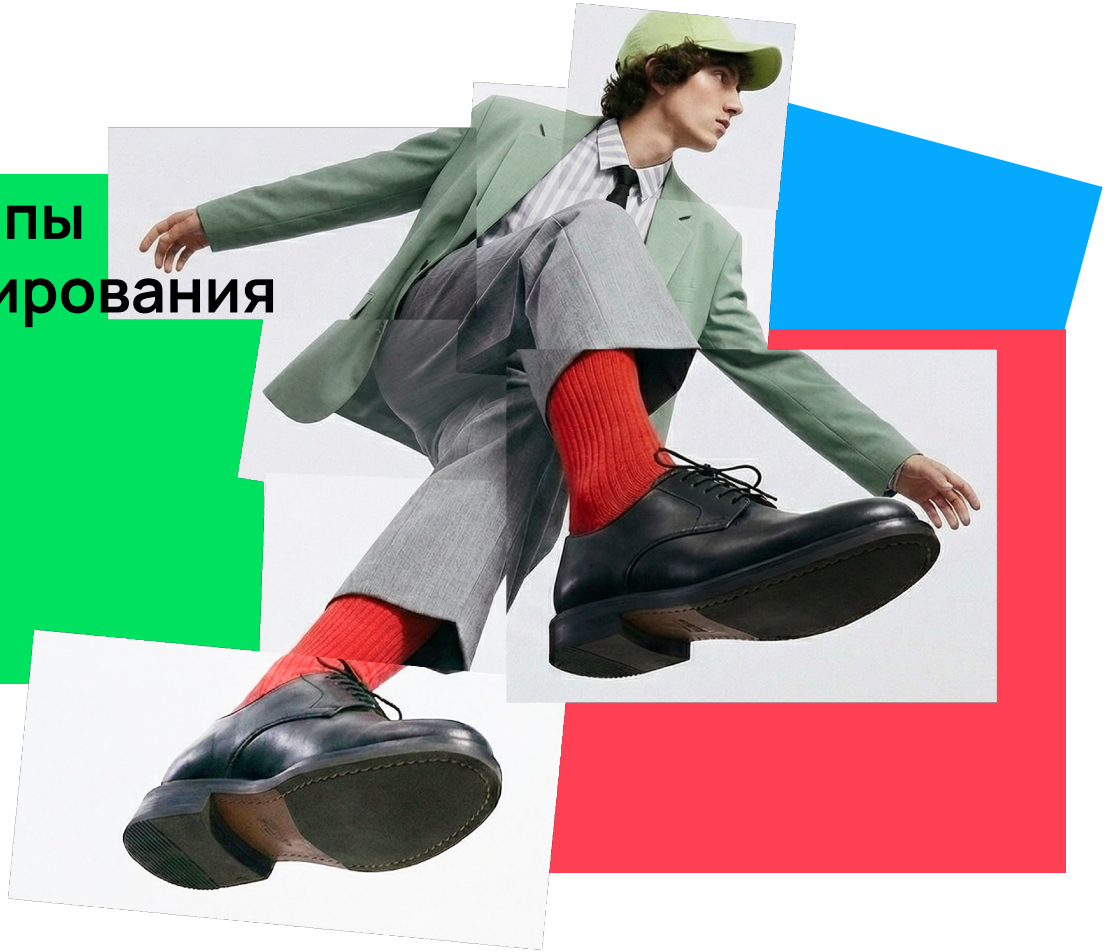
func (o Order) Total() int {
    total := 0
    for _, item := range o.items {
        total += item.pizza.Price() * item.count
    }
    return total
}
```



Avito Start


Q&A
time

Принципы проектирования кода



SOLID

S **Single Responsibility**
У модуля только одна причина для изменения

O **Open/Closed**
Предусматриваем расширение поведения без изменения стабильного кода

L **Liskov Substitution**
Каждая реализация соблюдает обещания общего контракта

I **Interface Segregation**
Потребитель зависит только от нужных ему операций

D **Dependency Inversion**
Бизнес-логика зависит от абстракций, детали реализуют эти абстракции

SRP: нарушение

Модуль должен объединять обязанности, которые **меняются по одной причине**. Обязанности с разными причинами изменения стоит разделять.

```
type Order struct {
  prices []int
}

func (o Order) Total() int {
  total := 0
  for _, price := range o.prices {
    total += price
  }
  return total
}

func (o Order) Receipt() string {
  return fmt.Sprintf("Итого: %d руб.", o.Total())
}
```

SRP: правильный пример

Order отвечает за состав и сумму заказа.

ReceiptFormatter отвечает за текст чека. Изменение оформления чека не меняет Order.

```
type Order struct {
  prices []int
}

func (o Order) Total() int {
  total := 0
  for _, price := range o.prices {
    total += price
  }
  return total
}

type ReceiptFormatter struct{}

func (ReceiptFormatter) Format(o Order) string {
  return fmt.Sprintf("Итого: %d руб.", o.Total())
}
```

ISP: нарушение

Потребитель должен зависеть только от тех операций, которые ему нужны.

```
type Pizza interface {
  Price() int
  Bake() error
  Deliver(address string) error
}

func ItemTotal(pizza Pizza, count int) int {
  return pizza.Price() * count
}
```

ISP: правильный пример

ItemTotal зависит только от Price().

Выпечка и доставка имеют отдельные контракты. Реализация может поддерживать несколько интерфейсов.

```
type Priced interface {
  Price() int
}

func ItemTotal(pizza Priced, count int) int {
  return pizza.Price() * count
}

type Baker interface {
  Bake() error
}

type Deliverer interface {
  Deliver(address string) error
}
```

OCP: нарушение

Если способы доставки регулярно добавляются, для каждого нового варианта приходится менять OrderTotal.

Здесь нужна точка расширения.

```
func OrderTotal(pizzaTotal, distanceKm int, pickup bool) int {
  if pickup {
    return pizzaTotal
  }

  return pizzaTotal + 200 + 20*distanceKm
}
```

OCP: правильный пример

Новый способ доставки реализует Delivery.

OrderTotal остаётся прежним. Точка расширения оправдана, когда новые способы действительно нужны.

```
type Delivery interface {
  Cost(distanceKm int) int
}
type Pickup struct{}
func (Pickup) Cost(int) int { return 0 }
type Courier struct{}
func (Courier) Cost(km int) int {
  return 200 + 20*km
}
func OrderTotal(total, km int, d Delivery) int {
  return total + d.Cost(km)
}
// OrderTotal не меняется.
type Express struct{}
func (Express) Cost(km int) int {
  return 400 + 30*km
}
```

LSP: нарушение

Контракт Flyer обещает полёт.

Penguin формально реализует интерфейс, но вызывает panic вместо выполнения обещанного поведения.

```
type Flyer interface {
  Fly()
}

type Sparrow struct{}

func (Sparrow) Fly() {
  fmt.Println("Воробей летит")
}

type Penguin struct{}

func (Penguin) Fly() {
  panic("Пингвин не умеет летать")
}
```

LSP: правильный пример

Flyer реализуют только летающие типы.

Penguin не обещает полёт. Launch может работать с любой реализацией Flyer, соблюдающей контракт.

```
// Контракт: Fly выполняет полёт.
type Flyer interface { Fly() }

type Sparrow struct{}
func (Sparrow) Fly() {
  fmt.Println("Воробей летит")
}

type Penguin struct{}
func (Penguin) Swim() {
  fmt.Println("Пингвин плывёт")
}

func Launch(bird Flyer) { bird.Fly() }
// Launch(Sparrow{}) допустим.
// Launch(Penguin{}) не скомпилируется.
```

DIP: нарушение

DIP: бизнес-логика зависит от абстракций, а детали реализуют их.

Здесь Pizzeria напрямую зависит от ElectricOven. Получение зависимости извне само по себе описывает DI, а не DIP.

```
func (p *Pizzeria) BakePizza() {
  oven := ElectricOven{}
  oven.Bake()
}
```

DIP: правильный пример

Pizzeria зависит от контракта Oven.
ElectricOven реализует этот контракт.

DIP задаёт направление зависимостей.
Передача Oven в конструктор является DI.

```
// Контракт принадлежит бизнес-логике.
type Oven interface { Bake() }
type Pizzeria struct { oven Oven }

func NewPizzeria(oven Oven) *Pizzeria {
  return &Pizzeria{oven: oven}
}
func (p *Pizzeria) BakePizza() {
  p.oven.Bake()
}

type ElectricOven struct{}
func (ElectricOven) Bake() {
  fmt.Println("Пицца выпекается")
}
// Сборка: NewPizzeria(ElectricOven{})
```

Другие принципы

1. KISS

Keep it simple, stupid

Выбирайте простое и понятное решение, которое удовлетворяет требованиям.

2. DRY

Don’t repeat yourself

Каждое бизнес-правило должно иметь единое авторитетное представление.

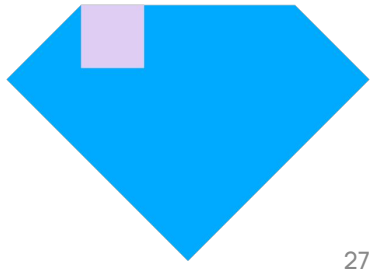
3. YAGNI

You aren’t gonna need it

Реализуйте возможности, когда они действительно нужны, опираясь на текущие требования.

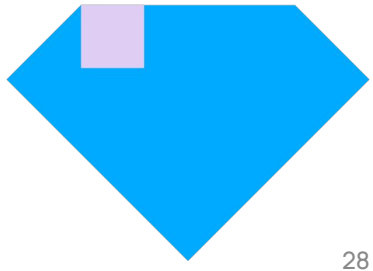
В пиццерии один способ доставки. Разработчик предлагает интерфейсы, фабрику и регистрацию плагинов на случай будущего расширения.

Как оценить предложение?



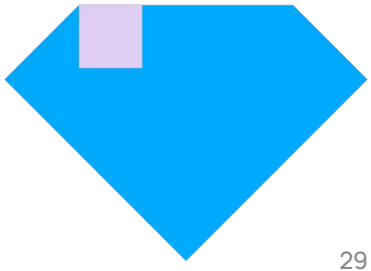
Поле количества пицц скрыто, но метод **SetQuantity** принимает любое число.

Достаточно ли этого для хорошей инкапсуляции? *



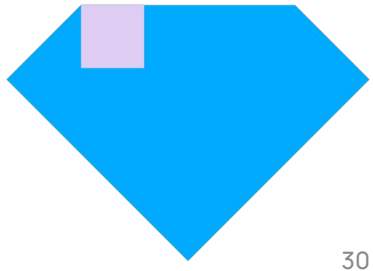
У заказа есть методы AddPizza, RemovePizza, ChangeQuantity и Total.

Нарушает ли он SRP, раз делает несколько вещей?*



Сервис получает ***FileLogger** через конструктор. Мы хотим проверять его в тестах без записи файлов.

Какие решения возможны?**




Q&A
time

GRASP шаблоны



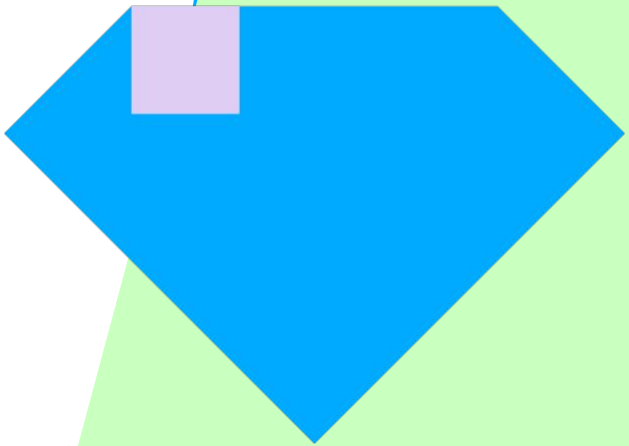
Avito Start

GRASP шаблоны

Принципы распределения ответственности



Отвечают на вопрос какому объекту или компоненту поручить обязанность



Важные шаблоны

01 High Cohesion	**03** Information Expert
02 Low Coupling	**04** Creator

Cohesion

Насколько тесно связаны обязанности **внутри модуля**

```
type Pizza struct {
  toppings []string
}

func (p *Pizza) AddTopping(name string) {
  p.toppings = append(p.toppings, name)
}

func (p Pizza) HasTopping(name string) bool {
  for _, topping := range p.toppings {
    if topping == name {
      return true
    }
  }
  return false
}
```

Coupling: высокая связанность

Код чека знает внутреннее устройство Pizza и сам вычисляет её цену.

Изменение состава или правил цены заставляет менять ReceiptTotal.

```
func ReceiptTotal(pizza Pizza) int {
  return pizza.Dough.Price +
    pizza.Cheese.Price +
    pizza.Sauce.Price
}
```

Coupling: правильный пример

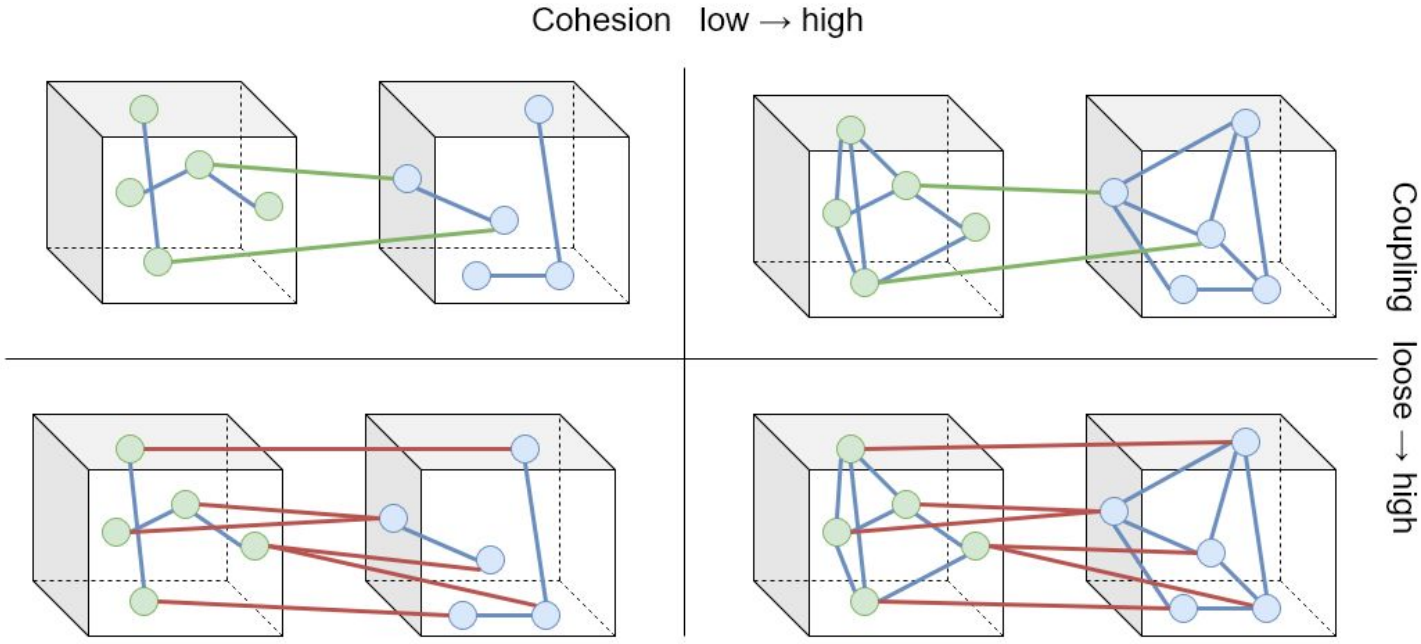
Код чека получает готовую цену через Price().

Состав пиццы и правила расчёта скрыты внутри Pizza. Их изменение не затрагивает ReceiptTotal.

```
type Ingredient struct { Price int }
type Pizza struct {
  dough, cheese, sauce Ingredient
}
func (p Pizza) Price() int {
  return p.dough.Price +
    p.cheese.Price +
    p.sauce.Price
}

type Priced interface { Price() int }
func ReceiptTotal(pizza Priced) int {
  return pizza.Price()
}
```

Coupling и cohesion



Information expert

Поручайте обязанность тому объекту, который располагает **необходимыми** для неё данными.

```
type OrderItem struct {
  pizza Pizza
  count int
}

func (item OrderItem) Subtotal() int {
  return item.pizza.Price() * item.count
}

type Order struct {
  items []OrderItem
}

func (order Order) Total() int {
  total := 0
  for _, item := range order.items {
    total += item.Subtotal()
  }
  return total
}
```

Creator

Поручайте создание объекта тому компоненту, который содержит его, тесно использует или располагает данными для его создания.

```
type OrderItem struct {
  pizza Pizza
  count int
}

type Order struct {
  items []OrderItem
}

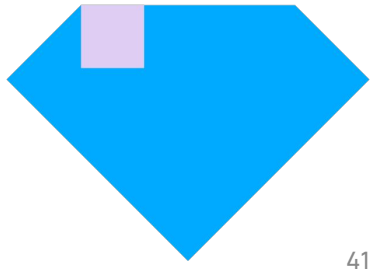
func (order *Order) AddPizza(pizza Pizza, count int) error {
  if count <= 0 {
    return errors.New("количество должно быть положительным")
  }

  item := OrderItem{
    pizza: pizza,
    count: count,
  }

  order.items = append(order.items, item)
  return nil
}
```

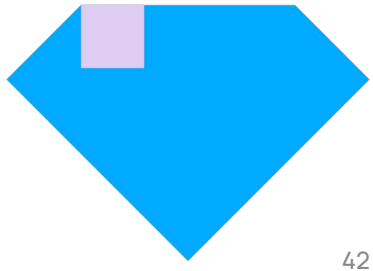
Код чека получает цену через pizza.Price(). Другой вариант — самостоятельно складывать стоимость теста, сыра и остальных ингредиентов.

Какой вариант уменьшает coupling и почему?



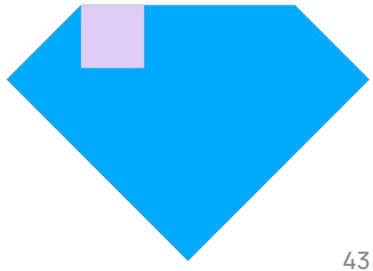
У заказа десять методов, и все работают с его составом и правилами изменения. У другого типа три метода: сохранить заказ в БД, отправить email и изменить размер изображения.

У какого типа выше cohesion?



Компонент располагает всеми данными для действия.

Достаточно ли этого, чтобы поручить действие именно ему?*




Q&A
time


Avito Start